

Nodrica Workflow-Orchestration Engine

A deterministic, domain-neutral TypeScript engine for validated node-based workflows

Project context: Reusable orchestration core for API, database, crawler, file, and AI workflows

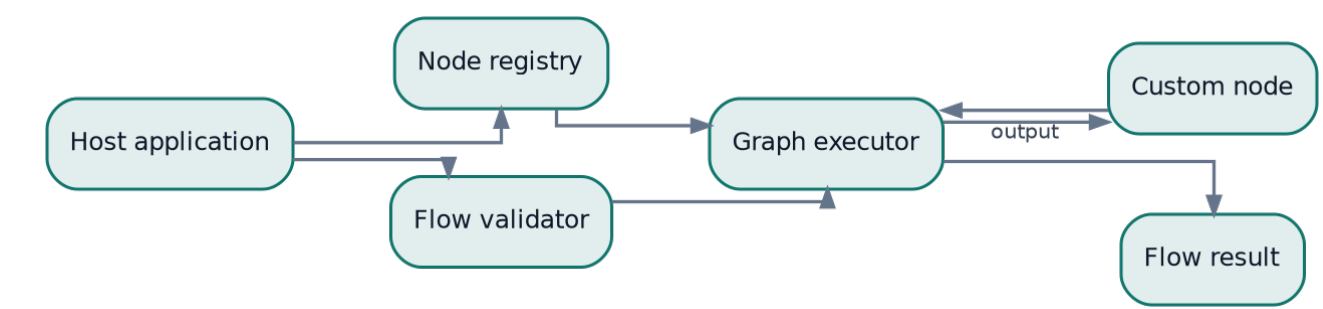
Status: Implemented TypeScript package

Nodrica lets a host register custom node types, define a directed acyclic flow, validate it before execution, run a deterministic path, capture node results, and apply explicit fallback, retry, timeout, cancellation, and event policies.

Technology stack

TypeScript	Directed acyclic graphs	Async handlers	Serializable contracts
Retries / backoff	AbortSignal	Unit tests	Package build

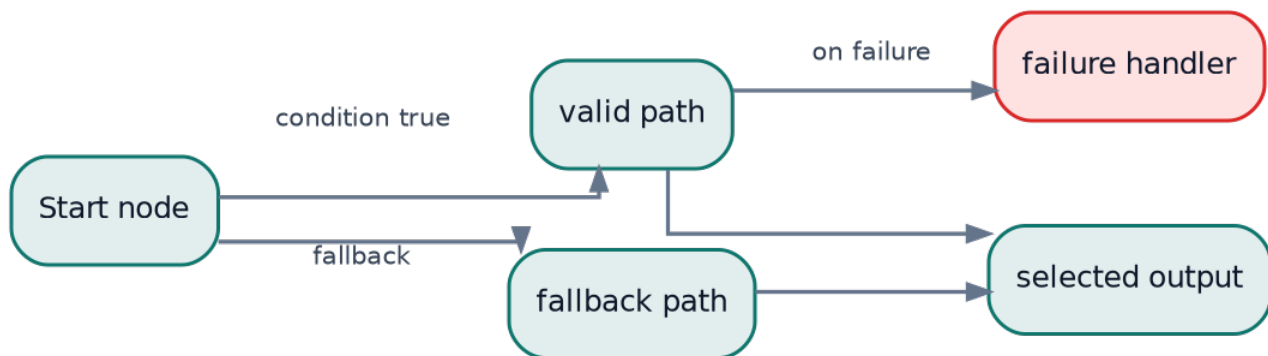
System at a glance



Graph validation Rejects missing nodes, cycles, disconnected graphs, and ambiguous policies.	Deterministic routing Evaluates conditional edges in declaration order with one fallback.
Failure handling Supports explicit failure edges after retry attempts are exhausted.	Execution controls Provides bounded retries, per-attempt timeouts, cancellation, and events.

System design and component ownership

The core never imports Gmail, databases, AI models, or crawlers. A host supplies those behaviors as registered node handlers, while Nodrica owns validation, graph traversal, state tracking, and result construction.



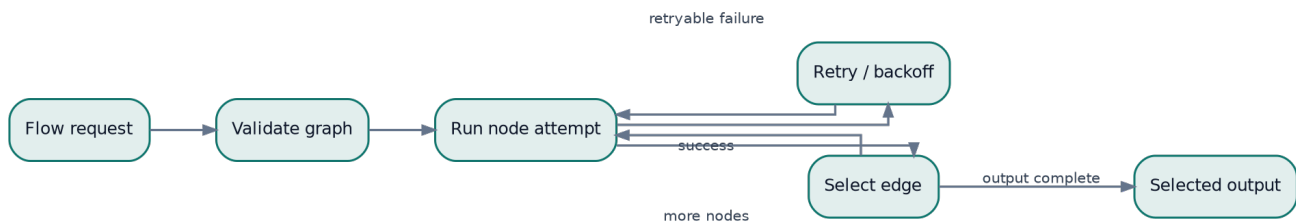
Component	Responsibility
Node registry	Maps a serializable node type to an asynchronous handler.
Flow validator	Checks graph shape, node identities, connectivity, and policy safety.
Graph executor	Selects and runs the deterministic next edge.
Node executor	Applies retry, backoff, timeout, cancellation, and result capture.
Event surface	Emits run and node lifecycle events without letting callbacks alter execution.
Result model	Returns selected output, per-node results, and serializable errors.

Key design decisions

- Conditions are data, not arbitrary functions, so flow definitions remain serializable.
- Only one branch executes in v2; parallel fan-out is deliberately excluded.
- Retries are opt-in and capped, with allowlisted error codes recommended.
- Checkpoint resume is not claimed until flow identity and idempotency are designed.

Core execution flow

A request is validated before any node runs. The executor starts at the single entry node, evaluates success or failure edges, records every result, and stops when the selected output completes or an unrecovered error occurs.



Implementation notes

- A flow must be a connected DAG with exactly one start node.
- Conditional edges are checked in declared order; one unconditional fallback may exist.
- A failure edge receives the failed node ID, type, input, error, and attempt count.
- Each handler receives an AbortSignal and attempt metadata.
- Observability callback failures are ignored so they cannot change workflow results.

Example registered flow

```
engine.registerNode({
  type: "uppercase",
  async run(input) {
    return { text: input.text.toUpperCase() };
  }
});

const result = await engine.run({
  input: { text: "hello" },
  nodes: [
    { id: "n1", type: "uppercase" },
    { id: "n2", type: "append", config: { value: " world" } }
  ],
  edges: [{ from: "n1", to: "n2" }],
  output: "n2"
});
```

Engineering review and production path

Engineering strengths

- Domain-neutral core with clean dependency direction.
- Validation-before-execution prevents unsafe partial runs.
- Deterministic semantics make behavior explainable and testable.
- Failure, retry, timeout, and cancellation policies are explicit.

Current limitations

- No parallel branch execution in the current version.
- No distributed workers or built-in persistent queue.
- Checkpoint resume is intentionally not implemented.
- Long-running handlers must cooperate with AbortSignal to release resources.

Recommended next steps



Why this is relevant to AI training work

- Directly demonstrates construction of nontrivial logical tasks and state transitions.
- Makes generated workflow behavior auditable instead of implicit.
- Encourages precise definitions of retries, conditions, failures, and selected output.
- Supports testing frontier-model code against deterministic orchestration contracts.

Repository:

<https://github.com/SidheshwarSarangal/nodrica>

Scope and provenance:

Prepared from the repository project reference, logic design, v2 orchestration contract, API design, and test-plan documentation.

